Complexity and approximation

How does problem size affect the cost of solving a problem? Measure time using the input encoding length, then study decision problems, verification, and polynomial-time reductions.

Next study approximation algorithms for optimization problems, using feasible solutions, bounds, and counterexamples to understand their guarantees.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter roadmap

Decision problems → P and NP → reduction direction → NP-completeness → approximation ratios and algorithms.

Distinguish a reduction proving hardness from an algorithm producing an approximate solution. NP-completeness alone proves neither P ≠ NP nor an exponential lower bound.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Write the problem as a language

We start with input bit length and define P, NP, and polynomial-time many-one reduction. The goal is to reconstruct 3SAT $\leq_p$ CLIQUE, including both proof directions.

Prerequisites are Boolean formulas, undirected simple graphs, asymptotic bounds, and universal/existential quantifiers.

You should distinguish finding a certificate from checking one and identify the two obligations in an NP-completeness proof.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Separate decision, search, and optimization

Form	CLIQUE example	Output
Decision	Is there a clique of at least k vertices?	yes/no
Search	Find a k-clique if one exists	A vertex set or failure
Optimization	Compute the maximum clique size ω(G)	An integer

P and NP here are classes of decision languages. A polynomial-time optimization algorithm decides the threshold version by comparing ω(G) with k. The converse requires an argument rather than a change of terminology.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Input length is not numerical value

Let x be the entire encoding and N = |x| its bit length. For $t \geq 1$, the binary encoding has $\lfloor \log_2 t \rfloor + 1$ bits. The graph representation and threshold k both count toward N.

For t = 2^m, the encoding needs m+1 bits. A loop with t iterations takes Θ(2^m) steps. Writing O(t) does not make this polynomial in the binary input length. A unary representation has t symbols and changes the encoding.

An adjacency matrix and an explicit adjacency list are polynomially interconvertible. This does not allow expanding binary integers into unary.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Input length is not numerical value (continued)

Same value, different input lengths

t = 2^m

Binary	Unary
1000...000	1111...111
m + 1 bits	2^m symbols
t loop steps = Θ(2^m)	t steps = Θ(input length)

Polynomial time is measured against complete encoding length N = |x|.

O(t) does not mean O(poly(log t)).

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Define P using encodings

A decision language L ⊆ {0,1}* contains the encodings of all yes instances. We reject malformed encodings in polynomial time.

$$L \in P \iff \exists M \ \exists p \text{ polynomial } \forall x: \ M(x) \text{ halts within } p(|x|) \text{ and } [M(x) = 1 \iff x \in L].$$

M is a deterministic decider. The bound covers every input, including no instances. PATH = {⟨G,s,t,k⟩ | an s-to-t path with at most k edges exists} is in P. Run BFS and compare its shortest distance with k.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

NP bounds certificate length

There are a deterministic verifier V and polynomials p and q such that

$$x \in L \iff \exists c,\ |c| \leq p(|x|) : V(x, c) = 1.$$

V halts in O(q(|x|+|c|)) time. The bound on certificate length makes verification polynomial in |x| for all permitted certificates. Reject overlong certificates first.

NP means nondeterministic polynomial time. It does not mean non-polynomial. The definition supplies a short certificate for a yes instance, without supplying a polynomial-time method to find it.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Completeness and soundness use different quantifiers

Obligation	Correct statement	Insufficient replacement
Completeness	$x \in L$ implies some short c is accepted	Every certificate is accepted
Soundness	$x \notin L$ implies every permitted c is rejected	Some certificate is rejected

V must halt on every input and permitted certificate. A validator that checks only syntax is insufficient. An NP verifier must check that the certificate establishes the yes answer.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Completeness and soundness use different quantifiers (continued)

Completeness and soundness of a verifier

Completeness

$x \in L$ → Some short certificate c → $V(x,c) = 1$

Existence does not mean every certificate succeeds.

Soundness

$x \notin L$ → Every permitted certificate c → $V(x,c) = 0$

Finding just one rejected certificate is insufficient.

$|c| \leq p(|x|)$; time $\leq q(|x| + |c|)$, for polynomials p and q.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Make the CLIQUE verifier executable

The input is a simple undirected graph G and a binary nonnegative k. Reject immediately if k > n. Accept an empty certificate for k = 0. Otherwise a certificate lists exactly k distinct vertex identifiers.

1. Check the encoding, certificate length, identifier range, and distinctness.
2. For every $i < j$, check $\{c_i, c_j\} \in E$.
3. Accept only if every check succeeds.

Certificate length is $O(k \log(n+1)) \leq O(n \log(n+1))$. With an adjacency matrix, $O(n^2+k^2)$ basic graph operations suffice; bit cost is also polynomial in input length. A clique supplies a valid list. Acceptance proves that every selected pair is adjacent.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

$P \subseteq NP$ is known; equality is unresolved

If $L \in P$, define $V(x,c)$ to run the decider for L and ignore c. The empty certificate works for every yes instance, and no certificate makes a no instance pass. Hence $P \subseteq NP$.

Whether $NP \subseteq P$, equivalently $P = NP$, remains unresolved. A diagram separating P from NP-complete languages requires an explicit $P \neq NP$ assumption.

Known NP-completeness theorems do not prove $P \neq NP$. An exponential brute-force algorithm does not prove an exponential lower bound for every algorithm.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A many-one reduction preserves yes and no

A $\leq$p B means that a total function f is computable in time polynomial in input length and, for every x,

$$x \in A \iff f(x) \in B.$$

The function must run in polynomial time; solving A cannot be assumed free. If A$\in$P and B is nonempty and not universal, decide A and output a fixed yes or no string of B accordingly. A many-one reduction outputs one string, rather than making multiple oracle queries.

The equivalence must hold for every string. If the source rejects malformed encodings and the target has a fixed no string, map them to it. A target need not have a no string: for example, B={0,1}*.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Read the direction through the solver

If $A \leq_p B$, a solver for B yields a solver for A. Compute x → f(x) → B-decider → the answer for A.

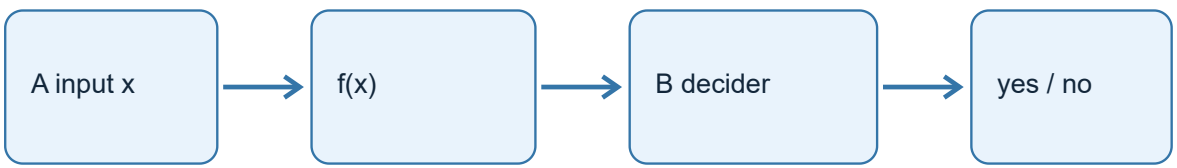
Thus $B \in P$ implies $A \in P$. Its contrapositive is that $A \notin P$ implies $B \notin P$. Neither $A \in P$ implies $B \in P$ nor hardness of B implies hardness of A.

To prove a target B hard, reduce a known hard A to B. Proving only $B \leq_p A$ shows that B can use a solver for A.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Read the direction through the solver (continued)

A ≤p B lets A use a solver for B



$x \in A \Leftrightarrow f(x) \in B$

$B \in P \Rightarrow A \in P$

To prove B hard, reduce a known hard A to B.

f must construct the target instance in polynomial time.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Track both time and output length

Suppose f runs in O(N^a) time and $|f(x)| \leq$ C N^a. If a B-decider takes O(M^b) time on an input of length M, the composite takes

$$O(N^a) + O((CN^a)^b) = O(N^a + N^{ab}).$$

a, b, and C are fixed constants, so this is polynomial. In the standard bit-output model, writing the output takes at least its length in steps.

A compressed description of an exponentially large B instance is not automatically an input to the original B-decider. Its original time bound cannot simply be reused.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Prove transitivity of reductions

Suppose $A \leq_p B$ through f and $B \leq_p C$ through g. Define $h = g \circ f$.

Semantically, $x \in A$ iff $f(x) \in B$ iff $g(f(x)) \in C$. Each equivalence matters.

If f takes O(N^a) time and g takes O(M^b), then $|f(x)| = O(N^a)$ and h takes $O(N^a + N^{(ab)})$ time. Therefore $A \leq_p C$.

This proves closure under a fixed number of compositions. If a transformation squares length at every one of N calls, the final length may be $N^{(2^N)}$. A polynomial number of calls alone is insufficient.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

NP-hard and NP-complete have different conditions

Term	Condition
B is NP-hard	For every $A \in NP$, $A \leq_p B$
B is NP-complete	B is NP-hard and $B \in NP$

If a known NP-hard A satisfies $A \leq_p B$, transitivity proves B NP-hard. A separate short-certificate verifier proves $B \in NP$ and completes the NP-completeness argument.

NP-hardness alone guarantees neither NP membership nor decidability. These definitions concern decision languages. For an optimization problem, specify the associated decision problem or the kind of solver reduction being used.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Construct 3SAT to CLIQUE

Let $F = C_1 \wedge \ldots \wedge C_m$, with three literal occurrences in each clause, allowing repetition here. The distinct-literal variant in CLRS is a restricted subset, so its NP-hardness also applies to this variant; assignment verification remains polynomial. We use the known NP-completeness of 3SAT from CLRS34 and prove the result for CLIQUE.

- Create one vertex v_{ij} for each occurrence, retaining its clause i and literal ℓ_{ij}.
- Add no edges within a clause.
- Between different clauses, add an edge exactly when the literals are not complements x and ¬x.
- Output $\langle G,k=m \rangle$.

Repeated copies of a literal remain distinct occurrence vertices. Equal literals in different clauses are consistent and must be adjacent.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Draw the complete graph of a concrete formula

Use $F = (x \vee \neg y \vee z) \wedge (\neg x \vee y \vee z) \wedge (x \vee y \vee \neg z)$.

Number occurrences by clause as 1:x, 2:¬y, 3:z; 4:¬x, 5:y, 6:z; 7:x, 8:y, 9:¬z. Each gray or green line in the figure is an actual edge.

The complete edge set is E = {15,16,24,26,34,35,36,17,18,19,27,29,37,38,48,49,57,58,59,67,68}, with 21 edges. Here ij denotes {i,j}, rather than multiplication.

For x=y=z=true, choose {1,5,7}. Edges 15, 17, and 57 all exist, giving a 3-clique.

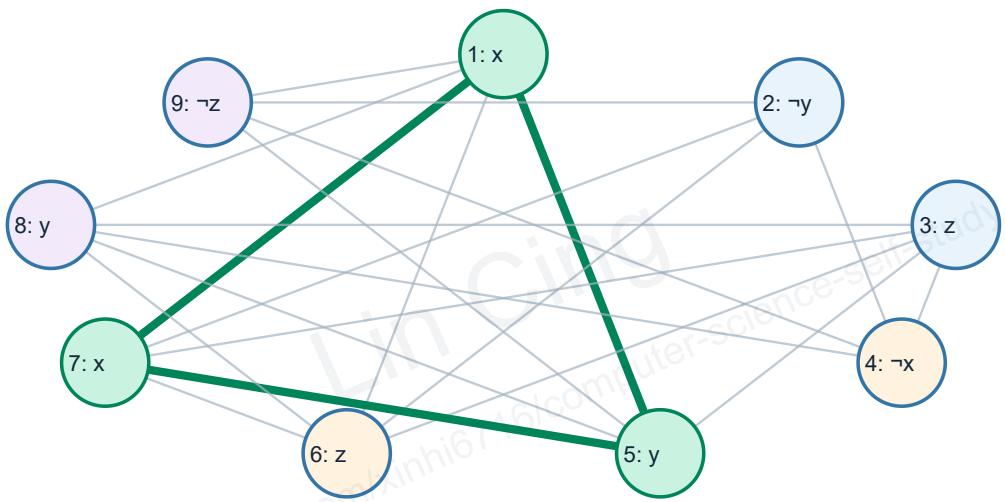
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Draw the complete graph of a concrete formula (continued)

3SAT → CLIQUE, all 9 vertices and 21 edges

F = (x ∨ ¬y ∨ z) ∧ (¬x ∨ y ∨ z) ∧ (x ∨ y ∨ ¬z)

C1 = {1,2,3} C2 = {4,5,6} C3 = {7,8,9}



Green {1,5,7} is a 3-clique. No internal or complementary edges.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Forward proof, assignment to clique

Assume F is satisfiable and fix a satisfying assignment. Choose one true literal occurrence from every clause. These choices give a set S of m vertices.

Any two chosen vertices belong to different clauses. Their literals are both true, so they cannot be complements x and ¬x. The construction therefore joins them by an edge.

Every selected pair is adjacent. Thus $|S|=m$ and S is an m-clique. This proves $F \in 3SAT$ implies $\langle G,m \rangle \in$ CLIQUE. The construction does not use the assignment; the correctness proof does.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Reverse proof, clique to assignment

Assume G has a clique S of size m. With no edges within a clause group, S contains at most one vertex from each group. There are m vertices and m groups, so S contains exactly one from each.

Every selected pair is adjacent, so selected literals contain no complementary pair. Set variables of selected positive literals to true and those of selected negative literals to false. No variable receives conflicting requirements. Assign all remaining variables arbitrarily.

Each clause contains its selected true literal, so F is satisfiable. This proves $\langle G,m\rangle \in$ CLIQUE implies $F \in$ 3SAT and completes the equivalence.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Keep cost and boundary conventions consistent

Input length N is at least proportional to the number of occurrences. For m three-literal clauses, the graph has 3m vertices and at most $9 \cdot m(m-1)/2$ edges. Pair comparisons, including bit comparisons of variable identifiers, and writing the edge identifiers take $O(N^2 \log(N+1))$ bit operations; an explicit edge list has $O(m^2 \log(m+1))$ bits, also polynomial in N.

Boundary	Convention and reason
m=0	The empty conjunction is true; the empty graph has a 0-clique
Opposite literals across clauses	Omit the edge to exclude conflicting assignments
A clause contains x and ¬x	Still no internal edges; select only one true occurrence
Repeated literal	Keep separate occurrence vertices
Malformed 3CNF	Output the fixed no instance ⟨empty graph,1⟩

If CLIQUE permits only $k \geq 1$, map m=0 to the fixed yes instance $\langle K_1, 1 \rangle$ instead.

Now conclude CLIQUE is NP-complete

1. Use the known NP-completeness of 3SAT.
2. Construct a polynomial-time total f and prove F satisfiable iff f(F) has an m-clique.
3. Thus 3SAT $\leq$p CLIQUE. Transitivity makes CLIQUE NP-hard.
4. The vertex-list certificate and verifier establish CLIQUE $\in$ NP.
5. Together these prove that CLIQUE is NP-complete.

If a polynomial-time CLIQUE algorithm were found, reductions would put every NP language in P. Together with P $\subseteq$ NP this would give P=NP. This conditional theorem does not prove P$\neq$NP.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Use a decision oracle to recover value and certificate

For a fixed graph G, the predicate that a clique of at least k vertices exists is monotone in k. Binary search on [0,n] finds ω(G) with O(log(n+1)) decision calls. This is a multiple-call solver procedure, rather than the single many-one mapping above.

Given a graph with a k-clique, try deleting each vertex v. Delete v when G−v still has a k-clique; otherwise retain it. After at most n calls the remaining graph is an inclusion-minimal yes instance and has exactly k vertices, which form a certificate.

If a vertex outside any chosen k-clique remained, it could still be deleted. A vertex that was indispensable cannot become dispensable after other vertices are deleted. Subgraph encodings do not grow. Thus a polynomial-time decision solver gives polynomial-time certificate recovery here.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

From a verifier to the first NP-complete problem

Given a Boolean circuit C, CIRCUIT-SAT asks whether some input y makes $C(y) = 1$. It is in NP: use the input assignment as a witness and evaluate gates in topological order in polynomial time.

For NP-hardness, fix an arbitrary language $L \in \mathrm{NP}$ and a deterministic polynomial-time verifier V with polynomial bounds p, T such that $x \in L \iff \exists y \in \{0,1\}^{p(|x|)} : V(x, y) = 1$. A length field and padding encode witnesses of shorter lengths.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

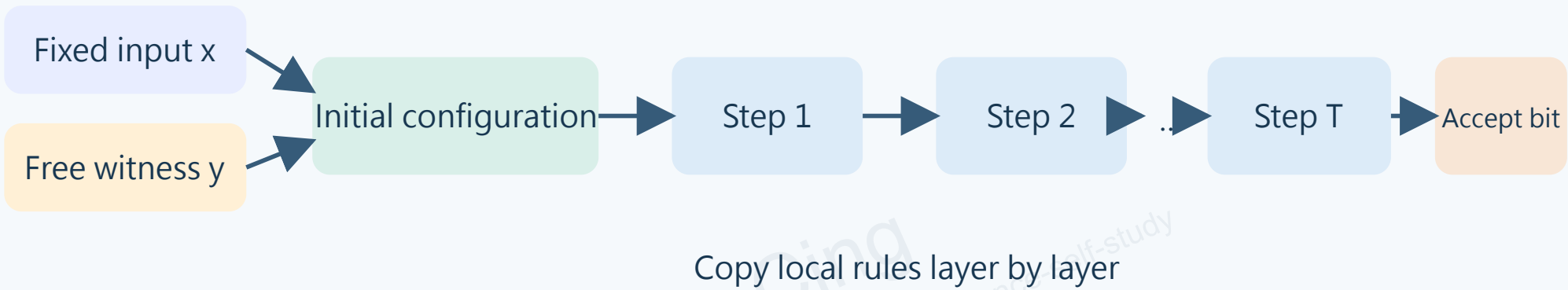
Unroll one execution into a circuit

Fix x and let $n = |x|$. Use one fixed encoding for all inputs. A configuration stores memory, head positions, and control state in $s(n) = \mathrm{poly}(n)$ bits. The verifier runs at most $T(n) = \mathrm{poly}(n)$ steps. Make the accepting state absorbing, so running exactly $T(n)$ steps preserves the answer.

In a single-tape Turing-machine model, each cell's next state depends only on itself and a fixed neighborhood. The finite local transition table has a constant-size Boolean gate implementation. Copy this circuit for every cell and every step, obtaining $O(T(n)s(n))$ gates. Other reasonable machine models must first be simulated with polynomial overhead; an arbitrary RAM operation is not automatically a constant-size gate.

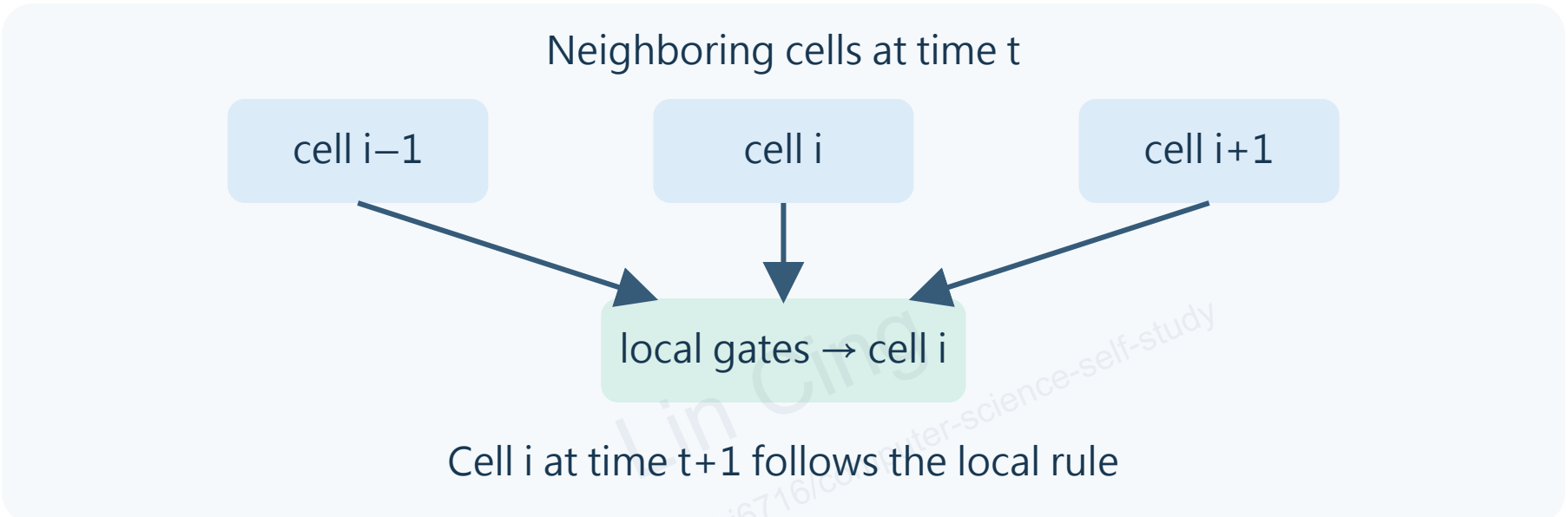
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Unroll one execution into a circuit (continued)



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

How local updates form the next layer



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Correctness by induction over configurations

The initial layer hardwires every bit of x, leaves the $p(n)$ witness bits as free inputs, and supplies blank cells and the initial control state. When witness lengths vary, the fixed-length encoding contains a length field and rejects invalid encodings.

For any free input y, layer 0 equals the initial configuration of $V(x, y)$. If layer t equals its actual configuration after t steps, the local gates implement the same transition and produce the configuration at step $t + 1$. By induction, the final output bit satisfies $C_x(y) = 1 \iff V(x, y) = 1$. Thus a satisfying circuit input exists exactly when $x \in L$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why the reduction takes polynomial time

Once L is fixed, its verifier, local rules, constants, and polynomial bounds are fixed too; the constructor may encode them. Given x, it computes $T(n), s(n)$, emits each layer of gates, and hardwires x. Its running time and output length are polynomial in n. This is a uniform construction, not merely a claim that a small circuit exists for each input length.

Therefore every $L \in NP$ has $L \leq_p \text{CIRCUIT-SAT}$. Together with membership, CIRCUIT-SAT is NP-complete. Wires may be shared; an exponentially large formula tree need not be produced. The reduction outputs the circuit, not a precomputed witness.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Circuit-to-SAT gate constraints

Give each wire value a variable, sharing the same gate-output variable where needed. Require output z_{out} by a unit clause. Encode every gate equivalently:

Relation	CNF constraints
z=u AND v	$(\neg z \vee u) \wedge (\neg z \vee v) \wedge (z \vee \neg u \vee \neg v)$
z=u OR v	$(z \vee \neg u) \wedge (z \vee \neg v) \wedge (\neg z \vee u \vee v)$
z=NOT u	$(z \vee u) \wedge (\neg z \vee \neg u)$

For AND, the first two clauses force both inputs true if z is true; the last forces z true if both inputs are true. OR and NOT follow by the same two-direction truth cases. Fix constant wires with unit clauses.

An accepting circuit input extends to all computed wire values and satisfies the constraints. Conversely, gate equivalences force every wire to equal its actual value in topological order, and the output unit clause forces acceptance. At most three clauses per gate give linear total size in gates and wires. Repeatedly expanding shared subcircuits into formula trees could instead cause exponential growth.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

SAT to three-variable clauses

Parse a formula into unary NOT and binary AND/OR nodes. Replace implication by $\neg a \vee b$. Other fixed-arity connectives have fixed-size truth-table gadgets. Give each internal node a new variable, apply the gate clauses, and require the root true. Remove repeated literals within a clause. Delete clauses containing both a literal and its negation, since they are tautologies.

Clauses now have at most three literals. To require exactly three distinct variables per clause, use fresh padding variables for each clause:

- Replace $(a \vee b)$ by $(a \vee b \vee p) \wedge (a \vee b \vee \neg p)$. If the original is true, any p works. If false, the pair forces both p and its negation.
- Replace unit a by the four clauses $(a \vee p \vee q), (a \vee p \vee \neg q), (a \vee \neg p \vee q), (a \vee \neg p \vee \neg q)$. If a is false, every p,q assignment falsifies one clause. If true, all pass.
- An empty clause is false, so output the padded version of $z \wedge \neg z$. An empty conjunction is true, so output one fixed satisfiable three-variable clause.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

SAT to three-variable clauses (continued)

Original assignments extend through parse-node values. Conversely, the tree constraints recover node semantics. Padding preserves each clause requirement. This is equisatisfiability, not literal equality of functions after adding variables. Constant clauses per node and at most four per padding step give polynomial size and construction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Splitting a long clause

For $l_1 \vee \cdots \vee l_r$, $r > 3$, a chain also works:

$$(l_1 \vee l_2 \vee y_1) \wedge (\neg y_1 \vee l_3 \vee y_2) \wedge \cdots \wedge (\neg y_{r-3} \vee l_{r-1} \vee l_r).$$

If all original literals are false, the first clause forces y1 true, each middle clause forces the next y true, and the last forces the final y false, a contradiction. If some literal is true, let yj indicate that the first j+1 literals are all false. Every chain clause then holds. This uses r-3 new variables and r-2 clauses. Unrestricted distributive expansion of a formula into CNF may be exponentially large.

For $F = (a \vee b) \wedge \neg a$, set $u = a \vee b, v = \neg a, z = u \wedge v$, add their gate clauses, and require z. Input a=false,b=true extends to u=v=z=true. If a=true, the NOT gadget forces v=false while z and AND require v=true, excluding a false acceptance.

The book's three-CNF definition uses three distinct literals. The main reductions here can further normalize to three distinct variables. Deduplication and padding also handle repeated literals before the occurrence construction.

Clique, independent set, and vertex cover

An independent set has no edge between its members. A vertex cover touches every edge. For a loopless undirected n-vertex graph, its complement has edge uv exactly when u≠v and G lacks that edge. Thus a set is a clique in G iff it is independent in its complement.

If S is independent in H, every H edge has an endpoint outside S, so $V \setminus S$ is a vertex cover. Conversely, a cover $V \setminus S$ forbids any edge with both endpoints in S, so S is independent. For $0 \le k \le n$,

$$\mathrm{CLIQUE}(G, k) \iff \mathrm{VC}(\overline{G}, n - k).$$

The reductions $(G, k) \mapsto (\overline{G}, n - k)$ and $(H, \ell) \mapsto (\overline{H}, n - \ell)$ both take $O(n^2)$ pair checks. A clique of at least k contains one of exactly k. A cover of at most n-k can be enlarged to exactly n-k. Handle out-of-range thresholds with fixed yes/no instances rather than negative certificate sizes.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A complement example

For vertices {1,2,3} and edges {12,23}, the set {1,2} is a size-2 clique. The complement has only edge 13. The same set is independent there, and its complement {3} covers the only edge, with size 3-2=1.

The transformation is to VC on the complement, with threshold n-k. Complementation is polynomial and reversible, giving reductions in both directions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Vertex cover to Hamiltonian cycle

A Hamiltonian-cycle certificate is a permutation of all vertices with valid consecutive edges and closure. This proves NP membership. For hardness, build an undirected gadget for every original edge {u,v}. Crossings in its drawing are not vertices, and no planar crossing gadget is required.

Create twelve vertices a1,...,a6 and b1,...,b6, where ai=[u,v,i] and bi=[v,u,i]. Add fourteen internal edges. Ten form the two chains a1-a2-a3-a4-a5-a6 and b1-b2-b3-b4-b5-b6. Four cross links are a1-b3, a3-b1, a4-b6, a6-b4.

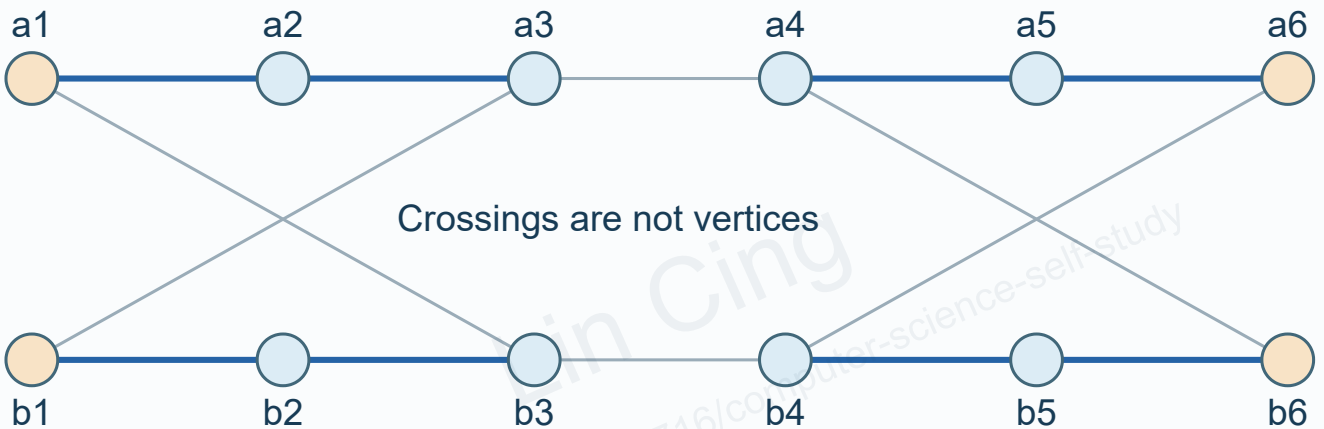
Only a1,a6,b1,b6 may connect outside the gadget. The other eight vertices need two internal edges in any final cycle. In particular a2,a5,b2,b5 already have degree two in the gadget, forcing their eight adjacent chain edges.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Vertex cover to Hamiltonian cycle (continued)

HC gadget: only a1,a6,b1,b6 connect outside

a_i=[u,v,i]; b_i=[v,u,i]



Eight blue edges are forced by degree 2 at a2,a5,b2,b5. Six gray edges remain undecided.

Middle edges a3−a4 and b3−b4 force the four cross links. Orange marks the four ports.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Three legal gadget covers

The remaining choices are the middle edges a3-a4 and b3-b4. Once these are selected or omitted, degree two at internal vertices forces every cross link.

Middle edges selected	Cover of all twelve vertices
Both	Two disjoint six-vertex paths, a1 through a6 and b1 through b6
Only b3-b4	One path a1-a2-a3-b1-b2-b3-b4-b5-b6-a4-a5-a6
Only a3-a4	The symmetric full path from b1 to b6
Neither	Two closed six-vertex cycles with no way to join the external cycle, forbidden

These cases exhaust all internal degree configurations. A legal Hamiltonian cycle uses one of the first three. It cannot enter an a port and leave through a b external chain. It must cover every gadget vertex. Locally, at least one endpoint of the original edge must be selected.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Middle edges force external port use

Let x indicate selection of a3-a4 and y selection of b3-b4. Vertex a3 already uses forced edge a2-a3, so a3-b1 has indicator 1-x. Similarly a4-b6 has 1-x. Edges a1-b3 and a6-b4 each have 1-y.

Thus a1,a6 have internal degree 2-y and external degree y. Ports b1,b6 have external degree x. Both ends on one original-vertex side connect externally together or neither does.

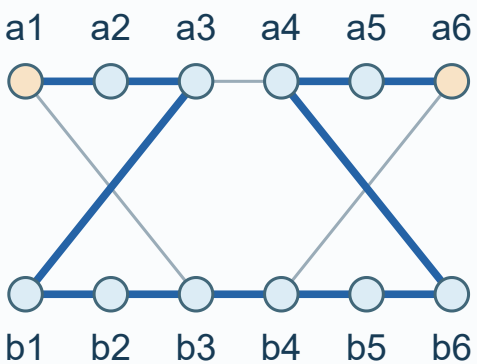
For x=y=0, the cycles are a1-a2-a3-b1-b2-b3-a1 and a4-a5-a6-b4-b5-b6-a4. Every vertex already has degree two, so neither cycle connects to a selector. They cannot form the whole graph's single Hamiltonian cycle. The three remaining full path covers are shown below.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Middle edges force external port use (continued)

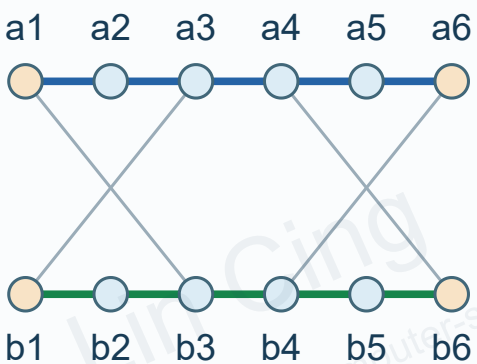
All three legal modes preserve the original-vertex side

u side only: 12 vertices



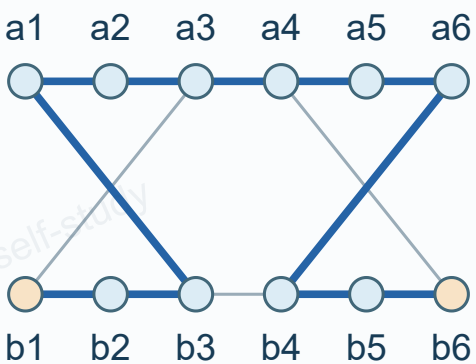
x=0, y=1

6 vertices on each side



x=1, y=1

v side only: 12 vertices



x=1, y=0

Gray edges are unused. Blue/green show paths. Each orange port needs one external edge for degree 2.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Incident chains and selectors

VERTEX-COVER asks for size at most k. Remove isolated vertices without changing the minimum cover. If k<0, or k=0 with an edge present, output a fixed no Hamiltonian-cycle instance. If no edges remain, or k is at least the remaining vertex count, output a fixed yes triangle. The main case has no isolated vertices and $1 \leq k < |V|$. A smaller cover can be enlarged to exactly k nonisolated vertices.

For each original u, order its neighbors u(1),...,u(d) arbitrarily. Connect its gadget sides by edges $[u, u(i), 6] - [u, u(i+1), 1]$, i=1,...,d-1. An external path selecting u must follow all of u's incident gadgets from one chain endpoint to the other.

Add k selectors s1,...,sk. Each selector connects to both ends of every u chain, with no other external edges. There are $12|E| + k$ vertices, $14|E|$ internal edges, at most $2|E|$ chain links, and $2k|V|$ selector links. The size is polynomial. Both source and target graphs are undirected.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Why activation propagates along the entire chain

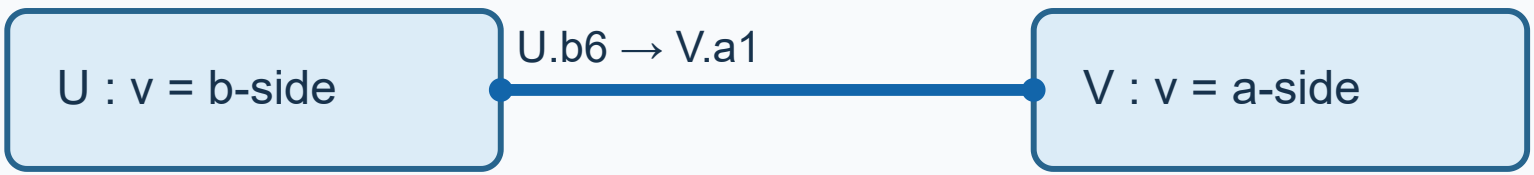
Call a u-side **active** when both of its ports use an external edge. Each interior port of an ordered u-chain has exactly one external edge available: the link to the adjacent u-side. If side i is active, it uses that link; the adjacent side's port then uses an external edge, so its other port must also do so because the two ports on one side are used together. Induction in both directions activates every incident gadget of u, including both chain endpoints. If one side is inactive, the same argument rules out any active side on that chain.

The two chain endpoints have only selector edges outside their gadgets. An active chain must therefore use one selector edge at each end. Its local paths connect its two u-ports, so following the cycle cannot switch to another original vertex before reaching a selector. An inactive side is nevertheless visited: its edge gadget has at least one active side, and when just the other side is active, that side's 12-vertex path covers every vertex, including the inactive side.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why activation propagates along the entire chain (continued)

Example: u–v–w, k=1, cover={v}



In this cover, v is active on both sides; U.b6 → V.a1 connects them

Unused u and w sides are visited inside the full 12-vertex paths

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Both directions of vertex cover to cycle

Given a cover $C = \{u_1, \ldots, u_k\}$, make one path through each selected vertex's incident chain. If only one endpoint of an original edge is selected, use that side's full twelve-vertex path. If both endpoints are selected, use the two disjoint six-vertex paths. Every edge has a selected endpoint, so every gadget vertex is visited exactly once. Join s1, the u1 chain, s2, the u2 chain, ..., sk, the uk chain, and back to s1. This visits every selector once and gives one Hamiltonian cycle.

Conversely, propagation along incident links makes each active u-side activate the entire u-chain, which connects two selectors and stays on the same original vertex. Select every active u. Each selector has cycle degree two and every selected chain consumes two selector incidences. Hence exactly k chains are active, including k=1 where both chain ends connect to the same selector by distinct edges. A chain cannot occur twice because it has only one pair of endpoints. In each edge gadget the zero-active case forms two isolated cycles, so at least one endpoint chain is active. The selected k vertices therefore cover every original edge.

Both directions of vertex cover to cycle (continued)

Therefore VC(G,k) holds iff the constructed graph has a Hamiltonian cycle. Construction, decoding, and size are polynomial. Fixed yes/no instances handle the preliminary cases without inventing empty chains for selectors.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A full twenty-five-vertex cycle

For original path u-v-w with k=1, choose cover {v}. Build U=W_{uv} and V=W_{vw}, each with twelve vertices, and one selector. The b side of U and a side of V represent v. Connect U.b6 to V.a1 and the selector to the two ends of the v chain.

Use these full paths, with every point prefixed by its gadget:

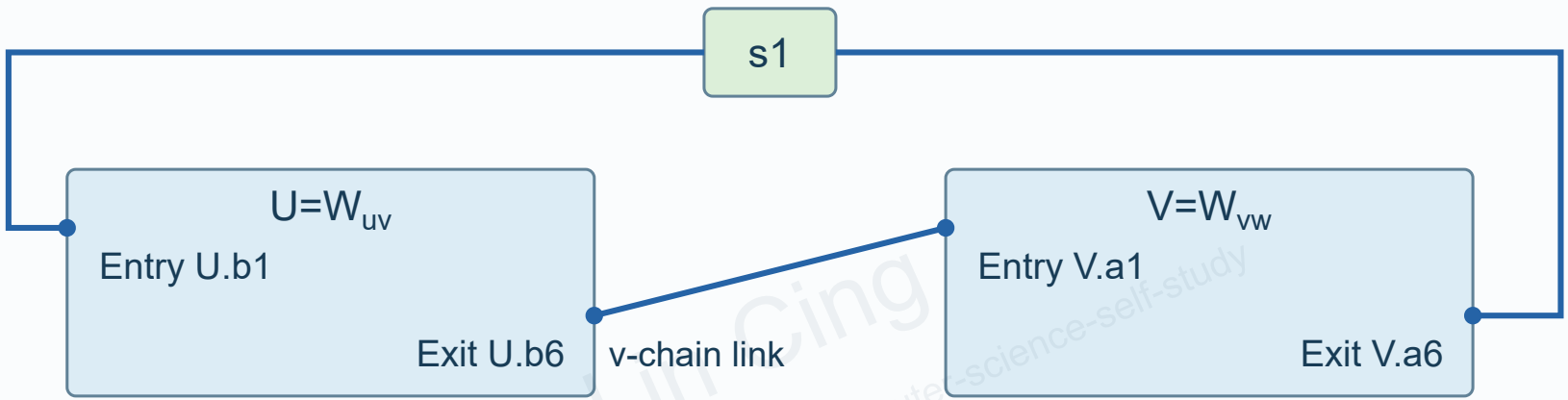
- U: b1,b2,b3,a1,a2,a3,a4,a5,a6,b4,b5,b6.
- V: a1,a2,a3,b1,b2,b3,b4,b5,b6,a4,a5,a6.

The complete cycle starts s1→U.b1, follows all of U, takes U.b6→V.a1, follows all of V, and returns V.a6→s1. Same-named vertices in different gadgets are distinct. Exactly 24 gadget vertices plus one selector are visited once. Every adjacency is an internal edge, chain link, or selector edge.

Both original edges have only v selected. In U, v is the b-side (x=1,y=0); in V, v is the a-side (x=0,y=1). The link U.b6–V.a1 forces both sides active; the other u and w sides are inactive, but their vertices appear inside these two full paths. Choosing only u would leave W_{vw} uncovered and cannot create a cycle.

A full twenty-five-vertex cycle (continued)

Original path u−v−w, cover={v}: one selector closes a 25-vertex cycle



Each box represents its full 12-vertex path. U and V vertices are distinct.

This shows the selected HC. All external edges follow the selector and incident-chain rules.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Hamiltonian cycle to TSP decision

Given an undirected graph with $n \geq 3$, build complete graph K_n. Give original edges weight 1 and missing edges weight 2. Set threshold B=n. Construction takes $O(n^2)$.

A Hamiltonian cycle uses n original edges and becomes a tour of cost n. Conversely, every tour has n edges of weight at least 1. Total cost at most n forces every edge to have weight 1, so every tour edge belongs to the original graph. It is a Hamiltonian cycle there.

CLRS uses weights 0 for original edges and 1 for missing edges with threshold 0. This teaching variant adds 1 to every edge: every n-edge tour gains exactly n, so its threshold becomes n and the yes/no answer is unchanged.

Thus HC(G) iff TSP(Kn,w,n). TSP decision is in NP. Its certificate is a vertex permutation, checked for uniqueness, closure, and total cost.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

TSP trace and small graphs

For a triangle with edges 12,23,31, all three weights are 1. Tour $1\to2\to3\to1$ costs 3 and meets threshold $B=3$. Removing original edge 31 leaves that edge in the complete target graph with weight 2. Every three-point tour then costs $1+1+2=4>3$, matching the absence of an original Hamiltonian cycle.

Simple undirected graphs with $n<3$ have no Hamiltonian cycle under this chapter's convention. Map them to a fixed no TSP instance. The n-edge argument above assumes $n\geq3$.

Optimization TSP returns a minimum cost and, when requested, a tour. A polynomial exact optimizer could decide the threshold problem by comparison, so hardness of decision also makes exact optimization hard unless P=NP.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Subset-sum membership and dynamic programming

Decision SUBSET-SUM asks whether some subset of given positive integers sums exactly to target t. A certificate lists selected indices. Check bounds, nonrepetition, and exact sum in polynomial time, proving membership in NP.

Let dp[i,s] mean that a subset of the first i integers reaches s. Then

$$dp[i, s] = dp[i - 1, s] \lor (a_i \leq s \land dp[i - 1, s - a_i]).$$

The $O(nt)$ algorithm is pseudopolynomial with binary t. The reduction below uses large values with polynomial bit length, consistently with that DP limitation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Three-CNF-SAT to a digit table

Normalize to n appearing variables and k clauses, each with three distinct variables and no complementary tautology. Deduplication and padding above achieve this. Map constant true or false formulas separately to fixed yes/no integer instances.

Create n+k decimal columns, first for variables and then clauses. These columns compactly define integers. The final delivered encoding is binary.

- For each xi, create Ti and Fi. Both have 1 in variable column i and 0 in other variable columns. Ti has 1 in each clause containing xi; Fi has 1 in each clause containing ¬xi. Other clause digits are zero.
- For each clause j, add two slack numbers, with digits 1 and 2 respectively in that clause column and zero elsewhere.
- The target has 1 in each variable column and 4 in each clause column.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Three-CNF-SAT to a digit table (continued)

There are 2n+2k positive numbers, each with n+k decimal digits and $O(n+k)$ binary bits. Total output length is $O((n+k)^2)$, and construction is polynomial. If the SUBSET-SUM input format requires a set rather than a multiset, the nondegenerate premises ensure Ti and Fi differ: each variable appears, and positive/negative occurrences cannot coincide in a clause. Different variables have different high columns, and slack columns also distinguish the slack numbers.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

No carry for every candidate subset

An arbitrary subset sums each variable column to at most 2. A clause has three distinct literals, so all truth-value numbers together contribute at most three 1s in its column. Slack adds at most 1+2=3. Thus every clause sum is at most 6<10. **No subset can carry**, not only a correctly constructed witness.

If the formula is satisfiable, choose exactly Ti or Fi according to each variable's value. Variable columns become 1. Each clause has r true literals, r=1,2,3. Fill its deficit 4-r by slack 1+2, 2, or 1, respectively. All columns reach the target.

Conversely, any subset reaching the target has exactly one of Ti,Fi per variable, because no carries can alter its variable column. These choices define a consistent assignment. Each clause target is 4 but its slack contributes at most 3, so some chosen truth-value number contributes 1 there. The corresponding literal is true. Every clause is satisfied.

The two implications and polynomial size prove NP-hardness. Together with NP membership, SUBSET-SUM is NP-complete.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A hand-checkable no-carry sum

Use $F = (x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee \neg x_3)$. Column order is x1,x2,x3,C1,C2.

Numbers	Decimal column representation
T1,F1	10010, 10001
T2,F2	01011, 01000
T3,F3	00110, 00101
S1,S1′	00010, 00020
S2,S2′	00001, 00002
Target	11144

Assignment (true,true,false) chooses T1+T2+F3=11122. Each clause has two true literals. Add S1′=20 and S2′=2 to reach 11144. No carries occur. Each high digit 1 confirms one truth-value number per variable.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A hand-checkable no-carry sum (continued)

Values can be of order 10^{n+k}, while binary length is only $O(n+k)$. The reduction has polynomial output even though exact $O(nt)$ DP is not polynomial in bit length.

T1+T2+F3=11122; add slack 00022 to reach target 11144

	x1	x2	x3	C1	C2
T1	1	0	0	1	0
T2	0	1	0	1	1
F3	0	0	1	0	1
slack	0	0	0	2	2
target	1	1	1	4	4

Each variable digit is 1

Each clause has 2 true literals

Slack 2 fills each to 4

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Why universal no-carry matters

Proving no carry only for satisfying-assignment selections would not establish the reverse direction. A wrong subset might otherwise forge target digits through carries. Here every subset has variable digits at most 2 and clause sums at most 6, so no carry can occur for any candidate.

Converting an n+k digit decimal number to binary by repeated multiplication by 10 and digit addition has a simple $O((n+k)^2)$ bit implementation. Performing it for all 2n+2k numbers is still polynomial. Verification sums at most $O(n+k)$ numbers of $O(n+k)$ bits, adding only $O(\log(n+k))$ bits to the sum.

For assignment (false,false,false) in the example, the two clauses have 0 and 2 true literals. Even both slack numbers give only 3 in the first clause column, below 4. No carry from another column can conceal the failure.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A guarantee quantifies over every input

An NP-hard optimization problem can still admit a polynomial-time approximation. Each algorithm must return a feasible solution and prove its relation to an unknown OPT on every allowed input. Good performance on one instance is not a ratio proof.

We use matching and MST lower bounds, element charges, and trimmed dynamic programming. Prerequisites are greedy algorithms, MSTs, induction, and binary input length.

Sources are CLRS 3e §§35.1–35.3 and 35.5, MIT 2004 lecture 24, and handwritten notes PDF 54–56.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Minimization and maximization reverse the bound

Let A be a feasible value, OPT the optimum, and $\rho \geq 1$. For positive objectives, $\max(A/OPT, OPT/A) \leq \rho$ unifies both directions.

Problem	Feasibility	Guarantee
Minimize	$OPT \leq A$	$A \leq \rho OPT$
Maximize	$A \leq OPT$	$A \geq OPT/\rho$

Minimization values OPT=10, A=15 and maximization values OPT=15, A=10 both give ratio 1.5. For maximization, $A \leq \rho OPT$ says almost nothing. Handle OPT=0 with multiplicative inequalities rather than division. Subset sum returns 0 if no positive item fits.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Vertex cover selects both endpoints

For an undirected graph $G=(V,E)$, a vertex cover C touches every edge; minimize $|C|$. Initialize $C=\varnothing$ and $R=E$. While R is nonempty, choose $uv\in R$, add u and v to C, and delete every edge incident to either endpoint.

Every deleted original edge has an endpoint in C, so the final C is feasible. A single edge scan can build a maximal matching and return all its endpoints in $O(|V|+|E|)$ time. This algorithm does not directly solve weighted vertex cover.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Vertex cover selects both endpoints (continued)

Vertex cover: two endpoints per round

a — b — c — d — e — f

1: ab → C={a,b}; R={cd,de,ef}

2: cd → C={a,b,c,d}; R={ef}

3: ef → |C|=6; |M|=3; OPT=3

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A round trace exposes the certificate

Use E={ab,bc,cd,de,ef} and choose ab, cd, ef in this order.

Round	Edge	Added vertices	Remaining edges deleted	Edges left
1	ab	a,b	ab,bc	cd,de,ef
2	cd	c,d	cd,de	ef
3	ef	e,f	ef	∅

M={ab,cd,ef} is a matching and C contains all six vertices. The cover {b,d,f} has size 3; the three disjoint edges force any cover to use at least 3 vertices. Thus OPT=3 and A/OPT=6/3=2, attaining the guarantee.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The factor 2 comes from a matching lower bound

Let M contain the selected edges. Selecting uv deletes every edge incident to u or v, so future selected edges share neither endpoint. Thus M is a matching.

Any cover C* must cover each edge of M. These edges have disjoint endpoints, so they require distinct cover vertices. Hence $|C^*| \geq |M|$. The algorithm takes exactly two new endpoints per selected edge, giving $|C| = 2|M| \leq 2|C^*|$.

M is maximal because no edge can be added at termination; it need not be maximum. On a-b-c-d, {bc} is maximal of size 1, while {ab,cd} is maximum of size 2. Taking only one arbitrary endpoint per matching edge can leave other edges uncovered.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Metric TSP specifies the graph and costs

The input is a complete undirected graph with at least three vertices and nonnegative symmetric costs satisfying $c(u,w) \le c(u,v)+c(v,w)$ for every triple. A feasible Hamiltonian tour visits each vertex once and returns to its start.

Compute an MST T and root it anywhere. A full DFS walk W traverses each tree edge once in each direction. Keep the order of first visits and close the sequence to obtain H.

W revisits the root and branch vertices, so it is a closed walk rather than a feasible TSP tour. Completeness ensures shortcut edges exist.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

From a tree walk to a numerical tour

Use Euclidean distances on A=(0,0), B=(1,0), C=(1,1), D=(0,1). An MST uses AB, BC, CD and costs 3.

The full walk A,B,C,D,C,B,A costs 6. Keeping first visits and closing gives A,B,C,D,A of cost 4. Every Hamiltonian cycle has four edges of length at least 1, so OPT$\geq$4; the displayed tour attains 4, proving OPT=4.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

From a tree walk to a numerical tour (continued)

Unit square: walk and tour

A(0,0), B(1,0), C(1,1), D(0,1)

T={AB,BC,CD}; c(T)=3

W=A,B,C,D,C,B,A; c(W)=6

H=A,B,C,D,A; c(H)=4=OPT

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The three steps of the double-tree proof

Delete any edge from an optimal tour H^* to obtain a spanning tree. Nonnegative costs prevent this deletion from increasing cost. MST optimality gives $c(T) \leq c(H^*)$.

W traverses each tree edge twice, so $c(W)=2c(T)$. Between consecutive retained vertices, let the original walk segment be $u=v_0,v_1,\ldots,v_k=w$. Repeated triangle inequalities give $c(u,w) \leq \sum_j c(v_{j-1},v_j)$. These segments partition W, so $c(H) \leq c(W)$.

Therefore $c(H) \leq c(W)=2c(T) \leq 2\mathrm{OPT}$. Dense Prim takes $O(n^2)$; walking and retaining first visits take $O(n)$, for $O(n^2)$ overall.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A shortcut can be more expensive without metric costs

Let AB=BC=CD=DA=1 and AC=BD=M>4 in a complete four-vertex graph. An MST can use AB, AD, BC rooted at A. Visiting D before B gives preorder A,D,B,C and tour cost 2+2M. The perimeter tour is optimal with cost 4.

The full walk A,D,A,B,C,B,A costs 6. Replacing D→A→B by D→B raises cost from 2 to M; replacing C→B→A by C→A also raises cost. The ratio (2+2M)/4 is unbounded. At M=10 it is 22/4=5.5.

This defeats the double-tree guarantee. The stronger impossibility of every constant approximation unless P=NP requires a separate gap reduction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Set cover chooses the most newly covered elements

Given universe U and family F with $\cup F = U$, choose as few sets as possible to cover U. Repeatedly select S maximizing $|S \cap R|$, where R is uncovered, until $R = \varnothing$. Break ties alphabetically.

Let $U = \{1,\ldots,6\}$, $A = \{1,2,3,4\}$, $B = \{1,2,5\}$, $C = \{3,4,6\}$. Greedy selects A for four new elements, B for element 5, and C for element 6. It uses 3 sets. $B \cup C = U$ and no set covers U alone, so $OPT = 2$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Set cover chooses the most newly covered elements (continued)

Charges at first coverage

A={1,2,3,4}; B={1,2,5}; C={3,4,6}

A → 1,2,3,4: each 1/4

B → 5: 1; C → 6: 1

Total=3; OPT=2; H4=25/12

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Split each unit cost among newly covered elements

If a round covers k new elements, charge each $c_x=1/k$. Each element is charged once, so the greedy set count is $|G|=\Sigma_x c_x$.

In the example, $c_1=c_2=c_3=c_4=1/4$ and $c_5=c_6=1$. The total is $4\times 1/4+1+1=3$. Set B carries charge 3/2, as does C.

For any optimal cover F*, each element occurs in at least one chosen set. Nonnegative charges imply $\sum_x c_x \leq \sum_{S\in F^*} \sum_{x\in S} c_x$. Counting overlap twice only increases the right side. It remains to bound the charge inside any one set.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The complete local harmonic charging argument

Fix any $S \in F$ with $s = |S|$. Order its elements $x_1, \ldots, x_s$ by first coverage, breaking ties within a round arbitrarily.

At the start of the round covering x_j, at least $s - j + 1$ elements of S remain uncovered. Earlier elements from the same round only make this lower bound weaker. The greedy set covers $k \geq |S \cap R| \geq s - j + 1$ new elements, since otherwise S would be a better choice.

Thus $c_{x_j} = 1/k \leq 1/(s - j + 1)$. Summing gives $\sum_{x \in S} c_x \leq \sum_{j=1}^{s} 1/(s - j + 1) = H_s$. This applies to every candidate set, including the sets in the optimal cover.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Local charges yield a global approximation ratio

If U=∅, greedy and optimal covers are empty and OPT=0. The cost inequality holds; do not evaluate 0/0, a maximum over an empty family, or ln 0. For U≠∅, set $H_0=0$ and $d = \max_{S\in F} |S| \geq 1$. Combining local charges with the optimal cover gives

$$|G| = \sum_x c_x \leq \sum_{S\in F^*} \sum_{x\in S} c_x \leq \sum_{S\in F^*} H_{|S|} \leq H_d |F^*|.$$

Since $H_d = 1 + \sum_{j=2}^{d} 1/j \leq 1 + \int_1^d dx/x = 1 + \ln d \leq 1 + \ln |U|$, greedy is an H_d approximation. Here d=4 and $H_4=25/12$; the bound is $3 \leq (25/12)\times 2 = 25/6$, while the actual ratio is 3/2.

A simple rescan takes O(mN) per round and at most N rounds, hence $O(mN^2)$ for m=|F| and N=|U|. This proves polynomial time. Do not drop the additive 1 in the logarithmic bound.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why exact subset-sum lists grow exponentially

Given positive integers $x_1,\dots,x_n$ and integer target $t \geq 1$, maximize a subset sum at most t. The empty subset is allowed. Exact states satisfy $P_0=\{0\}$, $P_i=P_{i-1} \cup (P_{i-1}+x_i)$, discarding values above t.

Every state corresponds to an actual subset, but there can be 2^n distinct states. A Boolean $O(nt)$ DP is only pseudo-polynomial because binary t has length $\Theta(\log t)$.

For $0<\varepsilon<1$, set $\delta=\varepsilon/(2n)$. Approximate lists retain separated representatives and aim for $\mathrm{OPT}/(1+\varepsilon) \leq \max L_n \leq \mathrm{OPT}$. Return 0 directly if $n=0$ or no item fits.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Trimming keeps smaller feasible representatives

In a sorted deduplicated list, keep the smallest value. Keep a later y only when $y>(1+\delta)\text{last}$, then update last; otherwise last represents y. Every original y has a retained z with $y/(1+\delta)\le z\le y$.

At round i, merge L_{i-1} and $L_{i-1}+x_i$, trim, then discard values above t. Always retain 0. Treat zero separately when bounding positive values with logarithms.

A smaller representative remains no larger than the corresponding true sum after adding the same nonnegative item. Retained values are actual subset sums. Store predecessor states and item indices if the subset itself is required.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Trimming keeps smaller feasible representatives (continued)

Trimmed lists: δ=0.05, t=308

L1={0,104}

L2={0,102,206}

L3={0,102,201,303}

L4={0,101,201,302}; OPT=307

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A round-by-round trimming calculation

Take x=(104,102,201,101), t=308, ε=0.4, n=4, δ=0.05. Use exact comparisons to avoid floating-point boundary errors.

i	Merged list	After trimming and removing >308
1	0,104	0,104
2	0,102,104,206	0,102,206
3	0,102,201,206,303,407	0,102,201,303
4	0,101,102,201,203,302,303,404	0,101,201,302

Round 2 removes $104 \le 102 \times 1.05 = 107.1$. Round 3 removes $206 \le 201 \times 1.05 = 211.05$. Round 4 removes $303 \le 302 \times 1.05 = 317.1$. The answer is 302=201+101; OPT=307=104+102+101. Thus $307/302 \approx 1.01656 \le 1.4$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

An induction tracks multiplicative error across rounds

Claim that each $y \in P_i$ with $y \le t$ has $z \in L_i$ satisfying $y/(1+\delta)^i \le z \le y$. The base case $i=0$ contains only 0.

If y omits x_i, use its previous representative q. If $y=p+x_i$, a representative q of p satisfies $q+x_i \ge p/(1+\delta)^{i-1}+x_i \ge (p+x_i)/(1+\delta)^{i-1}$ because $x_i \ge 0$. In both cases the merged list contains $w \in [y/(1+\delta)^{i-1}, y]$.

Trimming retains $z \in [w/(1+\delta), w]$, proving the claim. Since $z \le y \le t$, capacity deletion does not remove z. Applying the claim to OPT yields $A \ge \mathrm{OPT}/(1+\delta)^n$.

Using $\ln(1+\delta) \le \delta$ gives $(1+\delta)^n \le e^{\varepsilon/2} \le 1+\varepsilon$. For the last inequality, $f(\varepsilon)=\ln(1+\varepsilon)-\varepsilon/2$ has $f(0)=0$ and $f'(\varepsilon)=1/(1+\varepsilon)-1/2 \ge 0$ for $0 \le \varepsilon \le 1$. Thus the multiplicative guarantee $A \ge \mathrm{OPT}/(1+\varepsilon)$ holds in all cases. If $\mathrm{OPT}=0$, feasibility gives $A=0$ and no ratio is taken. If $\mathrm{OPT}>0$, the bound implies $A>0$, so division yields $\mathrm{OPT}/A \le 1+\varepsilon$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

An FPTAS is polynomial in bit length and 1/ε

After trimming, consecutive positive values have ratio >1+δ and lie in [1,t]. Therefore $|L_i| \le 2+\ln t/\ln(1+\delta)$. Since $\ln(1+\delta) \ge \delta/(1+\delta)$ and $\delta=\varepsilon/(2n)<1/2$, $|L_i| \le 2+3n \ln t/\varepsilon$.

Write $K=O(1+n \log t/\varepsilon)$. Linear merging of sorted lists takes $O(K)$ arithmetic operations per round, or $O(nK)$ overall. General sorting adds $O(\log K)$ but remains polynomial.

Binary input length B includes every x_i, t, and the rational encoding of ε. Retained integers use $O(\log t)$ bits; merged values are at most $t+\max x_i$. Rational threshold comparisons use cross multiplication, so bit complexity is polynomial in B and $1/\varepsilon$. Do not treat ε as a free infinite-precision real.

A PTAS needs polynomial time in B for fixed ε, such as $B^{1/\varepsilon}$; an FPTAS also needs polynomial dependence on $1/\varepsilon$. Trimming meets the latter requirement.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Weighted set cover: price per new element

A finite universe U is covered by sets with positive rational costs. Let R be the uncovered elements. Repeatedly choose a set with minimum $w(S)/|S \cap R|$ among sets with a nonzero denominator. Return the empty solution for empty U; report infeasibility if the union misses an element.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Trace and counterexample

Let $U = \{1, 2, 3\}$, $A = U$, $B = \{1, 2\}$, $C = \{3\}$, with costs 9, 2, 1. Their prices are 3, 1, 1. If B is chosen first, charge each of its elements 1, then choose C and charge element 3 one unit. Total cost is 3. Sets B, C cost 3; any solution containing A costs at least 9, so OPT=3. Choosing most new elements selects A first. Replacing its cost by arbitrary M makes that rule's ratio unbounded.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Trace and counterexample (continued)

Weighted cover: sets and charges

A: {1 2 3} cost 9

B: {1 2} cost 2

C: {3} cost 1

Round 1: B | 2 / 2 = 1

1: charge 1 2: charge 1

Round 2: C | 1 / 1 = 1

3: charge 1

ALG = 3 | OPT = 3

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Charges give a local bound

When S newly covers k elements, charge each $w(S)/k$. Every element is charged exactly on first coverage, so charges sum to ALG. Fix a set A and order its elements $e_1, \ldots, e_t$ by first coverage. At the beginning of the round covering e_i, at least $t - i + 1$ elements of A remain uncovered. This also holds for any ordering of elements covered in the same round.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sum the local bound over an optimal cover

Greedy could choose A in that round, hence $charge(e_i) \le w(A)/(t-i+1)$. Summing gives $\sum_{e \in A} charge(e) \le w(A) H_{|A|}$. Let $d = \max_S |S|$. An optimal cover contains each element at least once. Charges are nonnegative, so counting overlaps again only increases the right side:

$ALG \le \sum_{A \in OPT} \sum_{e \in A} charge(e) \le H_d OPT$, where $H_d = \sum_{i=1}^{d} 1/i \le 1 + \ln d$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

MAX-3-CNF: an expected guarantee

Each clause has three distinct variables, each independently assigned a fair bit. Only one of eight assignments falsifies a clause, so it is satisfied with probability $7/8$. Let Y count satisfied clauses and m be the total. Linearity yields $E[Y] = 7m/8 \geq 7OPT/8$; clauses need not be independent. A single clause still fails with probability $1/8$, so this is not a guarantee for every run. Repeated literals $(a \vee a \vee a)$ have satisfaction probability only $1/2$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

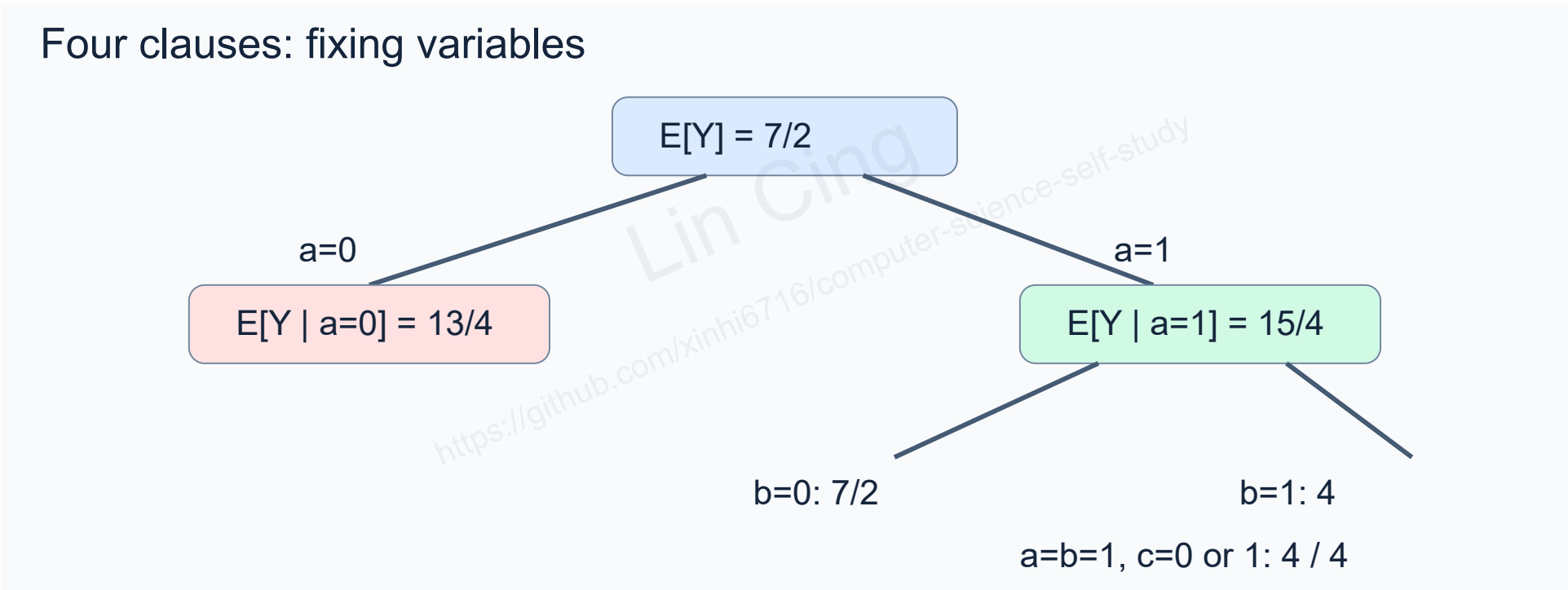
Fix variables without decreasing conditional expectation

Under a partial assignment, an already satisfied clause contributes 1, a completely assigned false clause contributes 0, and otherwise a clause with r unassigned variables contributes $1 - 2^{-r}$. The two conditional expectations after fixing the next variable to 0 or 1 average to the current expectation. Choose the larger and repeat. Once all variables are fixed, the expectation equals the actual satisfied count, at least $7m/8 \geq 7OPT/8$. Direct recomputation takes $O(nm)$ time.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Four-clause conditional-expectation trace

The clauses are $(a \vee b \vee c)$, $(\neg a \vee b \vee c)$, $(a \vee \neg b \vee c)$, $(a \vee b \vee \neg c)$. Initial expectation is $7/2$. Fixing $a = 0$ or 1 yields $13/4$ or $15/4$, so choose 1. Next $b = 0$ or 1 yields $7/2$ or 4, so choose 1. Either value of c satisfies all four.



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Weighted vertex cover: integer model and LP

An undirected graph has nonnegative rational vertex weights. Let $x_v \in \{0, 1\}$ denote selection. Minimize $\sum_v w_v x_v$ with constraint $x_u + x_v \geq 1$ for every edge uv. Relax to $0 \leq x_v \leq 1$. Every integer solution remains feasible, so $LP^* \leq OPT$. A polynomial-time rational LP algorithm gives an exact optimum.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Threshold rounding is feasible and at most twice optimal

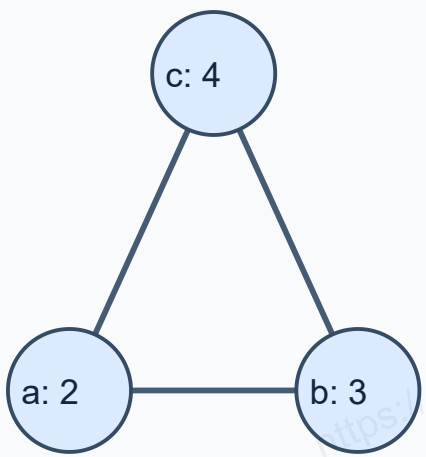
Select every vertex with $x_v \geq 1/2$. If both endpoints of an edge were omitted, their values would sum to less than 1, contradicting its constraint. Pointwise $1_{v \in C} \leq 2x_v$; multiplying by nonnegative weights and summing gives $w(C) \leq 2LP^* \leq 2OPT$. For one edge with endpoint weights 1 and M, taking both matching endpoints costs $1 + M$ while OPT=1. The unweighted rule cannot be reused unchanged.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Triangle: a lower-bound certificate and rounding

A triangle a, b, c has weights 2, 3, 4. The half-valued solution costs $9/2$. Multiply the constraints for ab, ac, bc by $1/2, 3/2, 5/2$ and add: every feasible LP costs at least $9/2$, so the half solution is optimal. Threshold rounding chooses all three at cost 9; the integer optimum selects a, b at cost 5, for ratio $9/5$.

Triangle LP and certificate



$x_a = x_b = x_c = 1/2$

ab * 1/2 + ac * 3/2 + bc * 5/2

$2x_a + 3x_b + 4x_c \geq 9/2$

LP* = 9/2 → threshold 1/2

{a,b,c}: cost 9 | {a,b}: OPT 5

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Randomized rounding: uncovered elements

Let $N = |U| \geq 2$, costs be nonnegative, and the optimal LP satisfy $\sum_{S \ni e} x_S \geq 1$ for every element. Set $k = 3\lceil \log_2 N \rceil$ and independently choose each set with $p_S = \min(1, kx_S)$. If some set containing e has $p_S = 1$, it is certainly covered; otherwise $\Pr(e \text{ uncovered}) = \prod_{S \ni e}(1 - p_S) \leq e^{-k} \leq N^{-3}$. A union bound gives probability at most $N^{-2} \leq 1/4$ that any element is uncovered.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

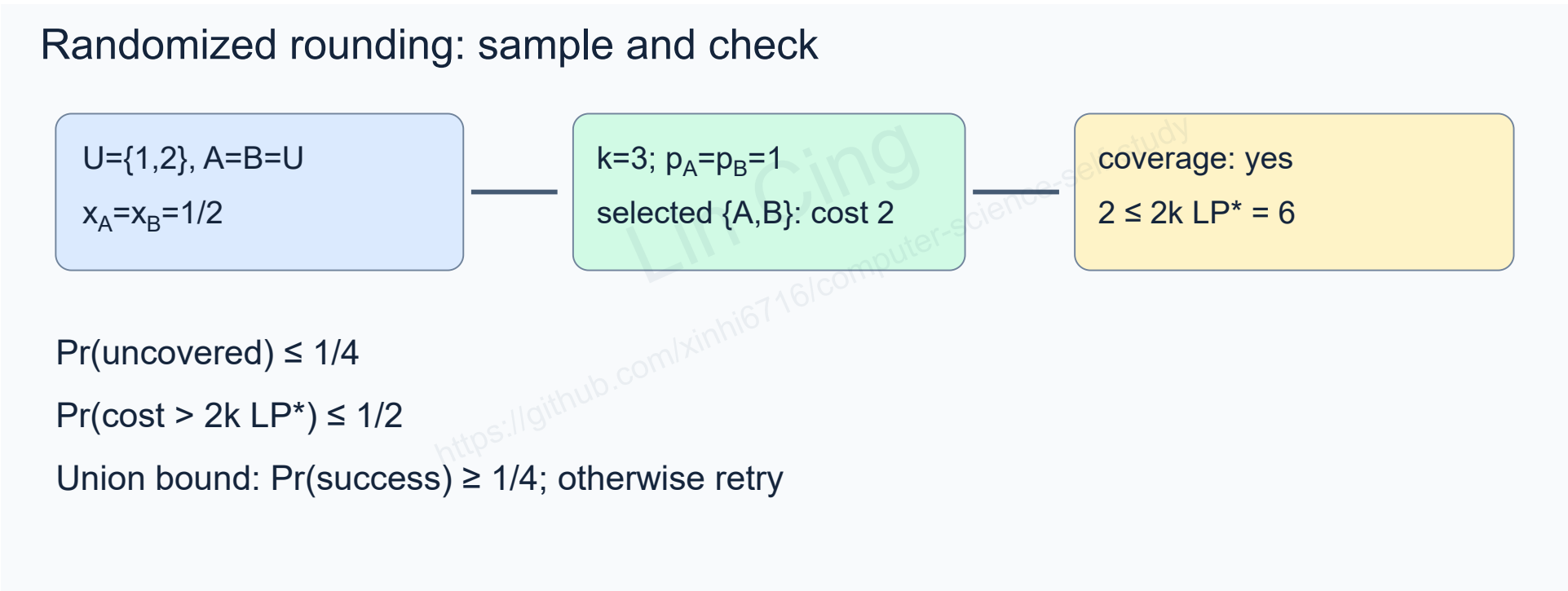
Check cost and coverage together

Expected cost $\sum_S w_S p_S \leq kLP^*$. If $LP^* > 0$, Markov bounds the probability that cost exceeds $2kLP^*$ by $1/2$. Adding the coverage failure bound gives probability at most $3/4$ that either condition fails, so success probability is at least $1/4$. Check both and retry independently after failure. Expected trials are at most 4; successful cost is $\leq 2kOPT$. If $LP^* = 0$, cost is necessarily zero and only coverage requires retrying. For $N = 0$ return empty; for $N = 1$ choose the cheapest covering set.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

One sampling step

Let $U = \{1, 2\}$, $A = B = U$, and each set cost 1. The optimal LP solution $x_A = x_B = 1/2$ has $LP^* = 1$. With $k = 3$, $p_A = p_B = 1$. Both are chosen, costing 2 and covering everything, with $2 \leq 2kLP^* = 6$. This illustrates clipping at 1; it is not tight for the failure bound.



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Nonmetric TSP: construct a gap

Assume a deterministic polynomial-time ρ-approximation for fixed $\rho \geq 1$, and choose a fixed integer $R \geq \rho$. Convert an n-vertex Hamiltonian-cycle input G into a complete graph: original edges cost 1, nonedges cost $M = Rn + 1$. If G has a Hamiltonian cycle, $OPT = n$ and the output costs at most Rn. Otherwise every tour uses a nonedge and costs at least $M + n - 1 > Rn$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

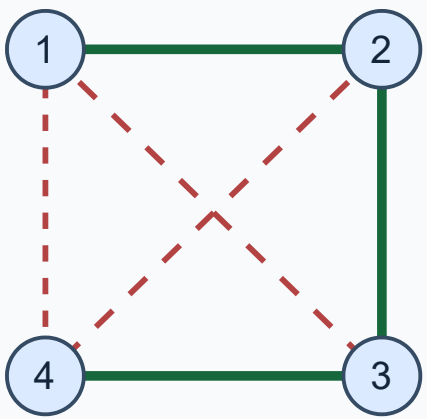
The cost threshold decides Hamiltonian cycle

Test whether the approximation output costs at most Rn. This decides Hamiltonian cycle, so existence of the approximation algorithm implies $P = NP$. The binary length of M is $O(\log n)$ and the complete graph has polynomial output size. For $n = 4, R = 2, M = 9$. If the original graph is a four-vertex path, each tour uses a nonedge and costs at least 12, above threshold 8. Triangle inequality can fail; this argument also does not directly rule out randomized algorithms with only expected guarantees.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The cost threshold decides Hamiltonian cycle (continued)

Path becomes a complete graph: cost gap



n=4, R=2, M=Rn+1=9

green original edge: 1

red dashed nonedge: 9

1-2-3-4-1: cost 12

no HC: every tour ≥ 12 > 8

HC: approximation output ≤ 8

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Three-literal clauses: expectation and success probability

Let $m \geq 1$ and let each clause contain exactly three distinct variables, possibly negated. Assign variables independently and uniformly. Exactly one of eight local assignments falsifies each clause, so $\mathbb{E}[X] = 7m/8$, where X counts satisfied clauses. Clauses may share variables: linearity of expectation does not require their satisfaction events to be independent.

Set $q = \Pr(X \geq 3m/4)$. Since $0 \leq X \leq m$, bound X by m on success and by $3m/4$ on failure:

$$\frac{7m}{8} = \mathbb{E}[X] \leq qm + (1-q)\frac{3m}{4}, \qquad q \geq \frac{1}{2}.$$

Since $OPT \leq m$, success also implies $X \geq 3OPT/4$. This is a probability lower bound, not a guarantee for every trial. The $7/8$ premise may fail for unit clauses or repeated variables.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Checkable retries and a small example

Run r independent trials and retain the assignment satisfying the most clauses. The chance of failing to reach $3m/4$ is at most 2^{-r}. Each trial can be checked by counting, without knowing OPT. For the single clause $(a \vee b \vee c)$, seven of eight assignments satisfy it, so $\Pr(X \geq 3/4) = 7/8$, above the general lower bound $1/2$. For $m = 0$, return any assignment directly.

This is a high-probability $3OPT/4$ guarantee. Expected approximation, single-trial success probability, and a guarantee for every run differ. Conditional expectations provide a separate deterministic method.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Christofides: objective and assumptions

The input is a complete undirected graph with $n \geq 3$, nonnegative symmetric edge costs, and triangle inequality $c(u, w) \leq c(u, v) + c(v, w)$ for all vertices. Distinct vertices may have zero distance. The goal is a minimum-cost Hamiltonian tour, with cost OPT. The algorithm returns a feasible tour costing at most $3OPT/2$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

From a tree to a connected even-degree multigraph

1. Find a minimum spanning tree T and its odd-degree vertex set O.
2. Find a **minimum-weight perfect matching** M in the complete graph induced by O.
3. Combine T and M as a multigraph, keeping both copies of any shared edge.
4. Find an Euler closed walk using every edge once. Skip repeated vertices and return to the start.

The handshake lemma makes $|O|$ even. The matching adds exactly one edge to each vertex in O, making its degree even; other degrees do not change. The graph stays connected because it contains T, and therefore has an Euler closed walk. Minimum-weight perfect matching in a general graph is polynomial-time solvable; an arbitrary matching does not guarantee the bound below.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

First lower bound: the tree costs at most the best tour

Removing any edge from an optimal tour gives a spanning tree. Nonnegative costs ensure this removal cannot increase cost, and the minimum spanning tree is no more expensive than that tree. Thus $c(T) \leq OPT$. This argument does not work unchanged with negative edges.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Second lower bound: extract the odd vertices from the best tour

Visit O in the order in which its vertices occur on an optimal tour, shortcutting every other vertex. Completeness supplies each shortcut edge, and repeated triangle inequalities yield a closed cycle on O costing at most OPT. A tree with $n \geq 3$ has at least two odd vertices. When $|O| = 2$, regard the cycle as two copies of the edge, one for each direction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Alternate the cycle edges into two perfect matchings

The number of odd vertices is even. Alternately color the edges of the cycle in two groups. Each group meets each odd vertex once and is a perfect matching. Their costs sum to at most OPT, so the cheaper costs at most $OPT/2$. The minimum matching M is no more expensive, giving $c(M) \leq OPT/2$. For two vertices, each of the two traversals gives the same single-edge matching, so the argument still holds.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Shortcutting preserves feasibility and does not increase cost

The Euler closed walk costs exactly $c(T) + c(M)$. Keep each vertex on its first occurrence and replace skipped stretches by direct edges. Triangle inequality ensures each shortcut does not increase cost. Return to the starting vertex to obtain a tour visiting every vertex once. Consequently

$$c(\text{tour}) \leq c(T) + c(M) \leq OPT + OPT/2 = 3OPT/2.$$

If OPT is zero, this multiplicative inequality still holds without dividing by OPT.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

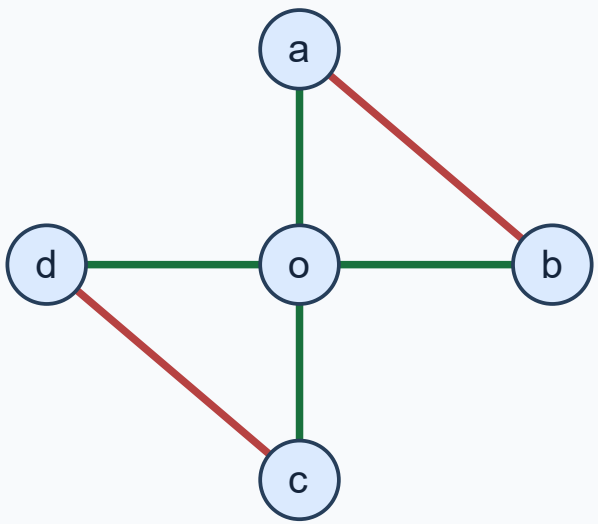
Four-leaf star metric: tree, matching and tour

There are five vertices o, a, b, c, d: center-to-leaf distance is 1 and every leaf-to-leaf distance is 2. These are tree-path distances and satisfy triangle inequality. The four star edges cost 4 and form an MST: any spanning tree has four edges, each costing at least 1. The center has degree 4 and each leaf degree 1, so $O = \{a, b, c, d\}$. All leaf pairs cost 2; choose $M = \{ab, cd\}$ for cost 4.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Four-leaf star metric: tree, matching and tour (continued)

Star MST, matching and Euler walk



Green: MST, four edges of cost 1

Red: matching ab, cd, each cost 2

Euler: o → a → b → o → c → d → o

Cost 1+2+1+1+2+1 = 8

Shortcut: o → a → b → c → d → o

Tour cost 8 = OPT

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The Euler walk and OPT in the example

One Euler walk is $o \to a \to b \to o \to c \to d \to o$, costing $1 + 2 + 1 + 1 + 2 + 1 = 8$. Skip the second occurrence of o to obtain $o \to a \to b \to c \to d \to o$, costing $1 + 2 + 2 + 2 + 1 = 8$. Every five-vertex tour uses two center edges of cost 1 and three leaf-to-leaf edges of cost 2, so OPT=8. This output is optimal; $3/2$ is a guarantee for all inputs satisfying the assumptions, not the actual ratio here.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd edition, chapters 34 and 35. MIT 6.046J Fall 2004, Lecture 24, approximation algorithms. Original handwritten algorithms notes, complexity and approximation pages.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study